

**COGNINET:
AI Powered Network Fault Detection
& Root Cause Analysis System**

**WIPRO NMS CAPSTONE PROJECT
A PROJECT REPORT**

Submitted by

230701138 — Kamalesh SP

230701158 — Kumaran R

230701514 — Ganesh M

**RAJALAKSHMI ENGINEERING COLLEGE
COMPUTER SCIENCE AND ENGINEERING
ANNA UNIVERSITY
OCTOBER 2026**

TABLE OF CONTENTS

CHAPTER 1 — INTRODUCTION.....	1
1.1 Background	1
1.2 Network Management System Context.....	1
1.3 Motivation.....	1
1.4 Problem Definition.....	2
1.5 Project Objectives	3
1.6 Scope	3
CHAPTER 2 — METHODOLOGY / SYSTEM DESIGN.....	4
2.1 Overall Methodology	5
2.2 Network Simulation and Device Environment	5
2.3 Network Device and Interface Management.....	6
2.4 Telemetry Collection and Processing.....	7
2.5 Alarm Management and Lifecycle	8
2.6 Backend and REST API Architecture	9
2.7 Database Design.....	10
2.8 Frontend and Dashboard	10
2.9 AI/RCA Architecture	10
2.10 Complete System Architecture.....	13
2.11 Implementation Flow	14
2.12 Methodology Summary.....	14
CHAPTER 3 — RESULTS AND DISCUSSION	15
3.1 Current Implementation Status.....	15
3.2 Network Device Management.....	15
3.3 Interface Monitoring	16
3.4 Telemetry Processing	17
3.5 Alarm Detection and Lifecycle	17
3.6 Backend API and Database Integration.....	18
3.7 Frontend Dashboard.....	19
3.8 AI/RCA Extension	19
3.9 System-Level Discussion	19
CHAPTER 4 — CONCLUSION AND FUTURE WORK	20
4.1 Conclusion.....	20
4.2 Future Work	20
4.3 Final Perspective	20
REFERENCES	21
APPENDICES.....	22
APPENDIX A — End-to-End Processing Flow.....	22
APPENDIX B — Key Terms.....	22
APPENDIX C — Core Data Model.....	23

ABSTRACT

The size and heterogeneity of modern computer networks have reached a level at which manual supervision of individual devices and interfaces is no longer a dependable operational practice. Network operators must continuously observe device status, interface state and resource utilisation, detect faults quickly, and maintain a reliable record of what has happened and when. When this information is distributed across device consoles and unrelated tools, fault detection is delayed, alarm handling becomes inconsistent, and the operator's view of the network remains incomplete. A centralised Network Management System (NMS) addresses these limitations by collecting operational information in one place and presenting it in a structured and consistent manner.

This report presents Cogninet, an intelligent Network Management System developed as part of the WIPRO NMS Capstone Project. Cogninet is designed to provide unified visibility and management of network devices, network interfaces, telemetry, operational status, faults and alarms. A controlled network environment is created using GNS3 and FRRouting (FRR), with the virtual routers R1, R2 and R3 acting as managed devices, and monitoring information is treated conceptually as SNMP / Telemetry. The core NMS is built on a layered Java 21 backend that uses Spring Boot 4.0.8, Spring Data JPA and Hibernate ORM, runs on Apache Tomcat 11 and exposes REST/JSON APIs. Persistence is provided by PostgreSQL 17 hosted on Supabase, with HikariCP managing database connections. A React and Tailwind CSS frontend presents devices, interfaces, telemetry and alarms through a centralised dashboard. The domain model is organised around four core entities: Device, NetworkInterface, Telemetry and Alarm.

A central function of the NMS is deterministic fault and alarm management. The operational state of an interface is evaluated using the `interface_oper_status` value, where 1 represents UP and 0 represents DOWN. When an interface is found to be DOWN, an alarm is created using the deterministic identifier `IF-DOWN-{deviceId}-{interfaceId}`. Repeated observations of the same condition update the existing active alarm instead of producing duplicates, the alarm is cleared when the fault is resolved, and a previously cleared condition can become active again. The alarm history is supported through the timestamps `firstSeenAt`, `lastSeenAt`, `clearedAt` and `createdAt`.

Building on this foundation, the report proposes an AI / Root Cause Analysis (RCA) extension that is kept clearly separate from the core NMS. The extension targets five capabilities: anomaly detection, root cause analysis, alarm correlation for noise reduction, failure prediction and incident explanation. Anomaly detection and prediction are proposed to rely on classical machine-learning techniques, root cause analysis on topology-aware reasoning over alarms and telemetry, and incident explanation on a Large Language Model (LLM) that serves as an explanation and decision-support layer rather than as the detector of faults.

A principal architectural feature of the proposal is a provider-independent AI/RCA layer. The same structured incident context can be supplied either to a locally hosted LLM through Ollama, which is suitable for privacy-sensitive network information and requires no external API key, or to an external LLM service through an authenticated API. The AI layer is intended to complement deterministic NMS logic and not to replace it, and its recommendations are to be validated by the network operator. The report distinguishes the core Cogninet NMS from the proposed AI extension throughout. No measured performance figures, prediction accuracies, datasets or experimental validation results are claimed for the AI capabilities; their implementation and evaluation are identified as future work. The report therefore describes a coherent NMS foundation together with a clearly defined architectural path from conventional monitoring and fault management toward intelligent network management.

CHAPTER 1

INTRODUCTION

1.1 Background

Computer networks form the communication backbone of enterprises, campuses and service providers. As these networks grow, they come to contain many routers, switches and links, each with several interfaces whose operational condition can change at any time. Interfaces may go down, devices may become heavily loaded, and traffic patterns may shift without warning. Each such change can affect the availability of the services that depend on the network, so the condition of the network must be observed continuously rather than inspected occasionally. Network management practice relies on several related activities. Monitoring collects operational information such as device status, interface status, processor and memory utilisation and traffic measurements; this information is commonly described as telemetry. Fault management interprets the collected information in order to detect abnormal conditions. Alarm management records these conditions, tracks them through their life and presents them to the operator, and visualisation brings the results together so that the operator can understand the state of the network quickly. The Simple Network Management Protocol (SNMP) [1] and the associated management information base [2] have long served as a standard foundation for obtaining monitoring information from network devices. This report introduces Cogninet, an intelligent Network Management System developed for the WIPRO NMS Capstone Project. Cogninet provides a centralised platform for managing devices and interfaces, processing telemetry, detecting faults, managing alarms and displaying network state in a dashboard. It is complemented by a proposed AI/RCA extension that is intended to help the operator interpret groups of related events.

1.2 Network Management System Context

A Network Management System brings the activities described above into a single platform. In Cogninet, the NMS context consists of device management, interface management, telemetry monitoring, fault management, alarm lifecycle handling, a centralised dashboard and a REST-based architecture. Device management maintains the identity and status of the managed devices. Interface management maintains the interfaces that belong to each device together with their operational state. Telemetry monitoring supplies timestamped observations of the network. Fault management evaluates these observations, and alarm lifecycle handling converts detected faults into alarms that are created, updated and cleared in a controlled way. The backend exposes the resulting information through REST/JSON APIs, which the dashboard consumes. To develop and demonstrate these functions without depending on a production network, Cogninet uses a controlled network environment built with GNS3 [3] and FRRouting (FRR) [4]. GNS3 provides the network simulation environment in which virtual devices are created and interconnected, while FRRouting provides routing functionality for those virtual devices. The simulated environment contains the virtual routers R1, R2 and R3. Because the environment is controlled, its behaviour can be observed repeatedly and the monitoring information produced by it can be used to exercise every layer of the NMS.

1.3 Motivation

The development of Cogninet is motivated by six practical problems that arise when networks are supervised without a centralised system.

1. Limited Network Visibility. Without a unified view, the operator cannot easily determine which devices and interfaces exist, which of them are operational and how they relate to one another. Faults in less frequently inspected parts of the network may remain unnoticed.

2. Manual Monitoring. Checking devices individually is time-consuming and does not scale with the number of devices and interfaces. Manual inspection is also periodic by nature, so a fault that appears between two inspections is detected only when the next inspection takes place.

3. Fragmented Network Information. Device status, interface state and performance measurements are often available in separate places. The operator must then combine them mentally, which makes it harder to relate an abnormal measurement to the device and interface that produced it.

4. Delayed Fault Detection. When faults are found only through manual checks or user complaints, the time between the occurrence of a fault and its detection increases, and the associated service disruption lasts longer.

5. Increasing Monitoring Complexity. As the numbers of devices, interfaces and measurements grow, the volume of information that must be examined grows with them, and the relationships among events become harder to see. The sixth problem leads directly to the motivation for intelligent analysis. A single underlying fault in a network can be accompanied by several related events, such as alarms reported for more than one interface or device. As the volume of events increases, an operator who must examine each event separately may find it difficult to separate the root cause from its downstream symptoms.

1.4 Problem Definition

The problem addressed in this project may be stated formally as follows. Given a network of managed devices, each with one or more interfaces, whose operational state changes continuously, design and implement a centralised Network Management System that (i) maintains an accurate inventory of devices and their interfaces, (ii) receives and processes telemetry and associates every observation with the correct device and interface, (iii) detects fault conditions from the processed telemetry, (iv) manages the resulting alarms through a controlled lifecycle without generating duplicate active alarms for the same condition, (v) retains historical information about devices, interfaces, telemetry and alarms, and (vi) presents the resulting information to the operator through a centralised dashboard. In addition, because a single fault may produce several related alarms, the system architecture must provide for event correlation and AI-assisted incident analysis. This analysis must be grounded in the structured data held by the NMS, must be separated from the deterministic fault-detection logic, and must not depend on a single AI provider. The problem is therefore to establish a dependable NMS foundation and to define an extensible, provider-independent AI/RCA layer on top of it.

1.5 Project Objectives

The objectives of the project are listed in Table 1.1. Objectives 1 to 9 concern the core Cogninet NMS, and objective 10 concerns the proposed AI/RCA extension.

Table 1.1 — Project Objectives

No.	Objective	Nature
1	Develop a centralised Network Management System that provides unified visibility of devices, interfaces, telemetry and alarms.	Core
2	Establish a controlled network environment using GNS3 and FRRouting with the virtual routers R1, R2 and R3.	Core
3	Implement device management through the Device entity.	Core
4	Implement interface management through the NetworkInterface entity and its association with devices.	Core
5	Process telemetry and associate each observation with the relevant device and interface.	Core
6	Persist devices, interfaces, telemetry and alarms in PostgreSQL 17 using Spring Data JPA and Hibernate ORM.	Core
7	Expose the managed information through REST/JSON APIs implemented in a layered Spring Boot backend.	Core
8	Implement alarm management with a deterministic alarm identifier, duplicate prevention, clearing and reopening.	Core
9	Provide a React and Tailwind CSS dashboard for devices, interfaces, telemetry and alarms.	Core
10	Define a provider-independent AI/RCA extension for anomaly detection, root cause analysis, alarm correlation, failure prediction and incident explanation, usable with a local Ollama LLM or a cloud LLM API.	Proposed extension

1.6 Scope

The scope of the core Cogninet NMS covers the following areas. The network simulation is based on GNS3 and FRRouting. Device management and interface management cover the Device and NetworkInterface entities and the association between them. Telemetry covers the receipt, processing and storage of operational information. Alarm management covers the detection of interface-down conditions and the complete alarm lifecycle. The backend is a Spring Boot application that exposes REST/JSON APIs, the database is PostgreSQL 17 hosted on Supabase, and the frontend is a React and Tailwind CSS dashboard.

The AI extension covers the design of an AI/RCA layer for anomaly detection, root cause analysis, alarm correlation, failure prediction and incident explanation, together with its provider-independent integration with a local Ollama LLM or an external LLM API. The AI capabilities are described at the architectural level. Their implementation, training, validation and quantitative evaluation require further work, and no experimental results for them are claimed in this report. Production-scale deployment, security hardening and large-scale performance testing are likewise outside the scope of the current stage.

CHAPTER 2

METHODOLOGY / SYSTEM DESIGN

2.1 Overall Methodology

The methodology adopted for Cogninet is layered. Each layer has a defined responsibility and passes its output to the next layer through a clearly identified interface, so that the network environment, the monitoring information, the backend, the database, the dashboard and the AI extension can be designed and discussed separately while still forming one continuous system. The core NMS follows the pipeline shown below.

GNS3 / FRRouting → SNMP / Telemetry → Cogninet Backend → Telemetry Processing → PostgreSQL → REST API → React Dashboard

The proposed AI extension is attached to this pipeline as a separate layer that consumes the data already held by the NMS:

Telemetry + Alarms + Topology + Historical Context → AI/RCA Engine → Local Ollama OR Cloud LLM API → Incident Intelligence

The technologies used in each layer are summarised in Table 2.1; the rows describing AI technologies are labelled as proposed.

Table 2.1 — Cogninet Technology Stack

Layer	Technology	Purpose
Network Simulation	GNS3	Network topology and virtual network environment
Routing	FRRouting	Routing functionality in virtual network devices
Monitoring	SNMP / Telemetry (conceptual)	Network operational information
Backend	Java 21	Application development
Backend Framework	Spring Boot 4.0.8	REST services and application framework
Persistence	Spring Data JPA	Data access
ORM	Hibernate ORM	Object-relational mapping
Build	Maven	Project and build management
Server	Apache Tomcat 11	Application runtime
Database	PostgreSQL 17	Persistent storage
Database Hosting	Supabase	PostgreSQL hosting
Connection Pool	HikariCP	Database connection management
Frontend	React	User interface
Styling	Tailwind CSS	User-interface styling
API	REST/JSON	Frontend/backend communication
AI Processing	Python (proposed)	Proposed AI/RCA processing

Layer	Technology	Purpose
Data Processing	Pandas, NumPy (proposed)	Proposed data preparation for AI/RCA
Machine Learning	Scikit-learn (proposed)	Proposed anomaly detection and ML processing
Local LLM	Ollama (proposed)	Proposed local LLM provider
Cloud AI	External LLM API (proposed)	Proposed cloud AI provider

Two principles govern the methodology. The first is that reliable operational detection is performed by deterministic NMS logic, namely the evaluation of telemetry, the determination of interface state and the generation and lifecycle management of alarms. The second is that the AI/RCA layer is an extension: it receives structured information from the NMS, produces analysis and explanation for the operator, and does not alter the deterministic detection path. The overall architecture that results from these principles is shown in Figure 2.1.

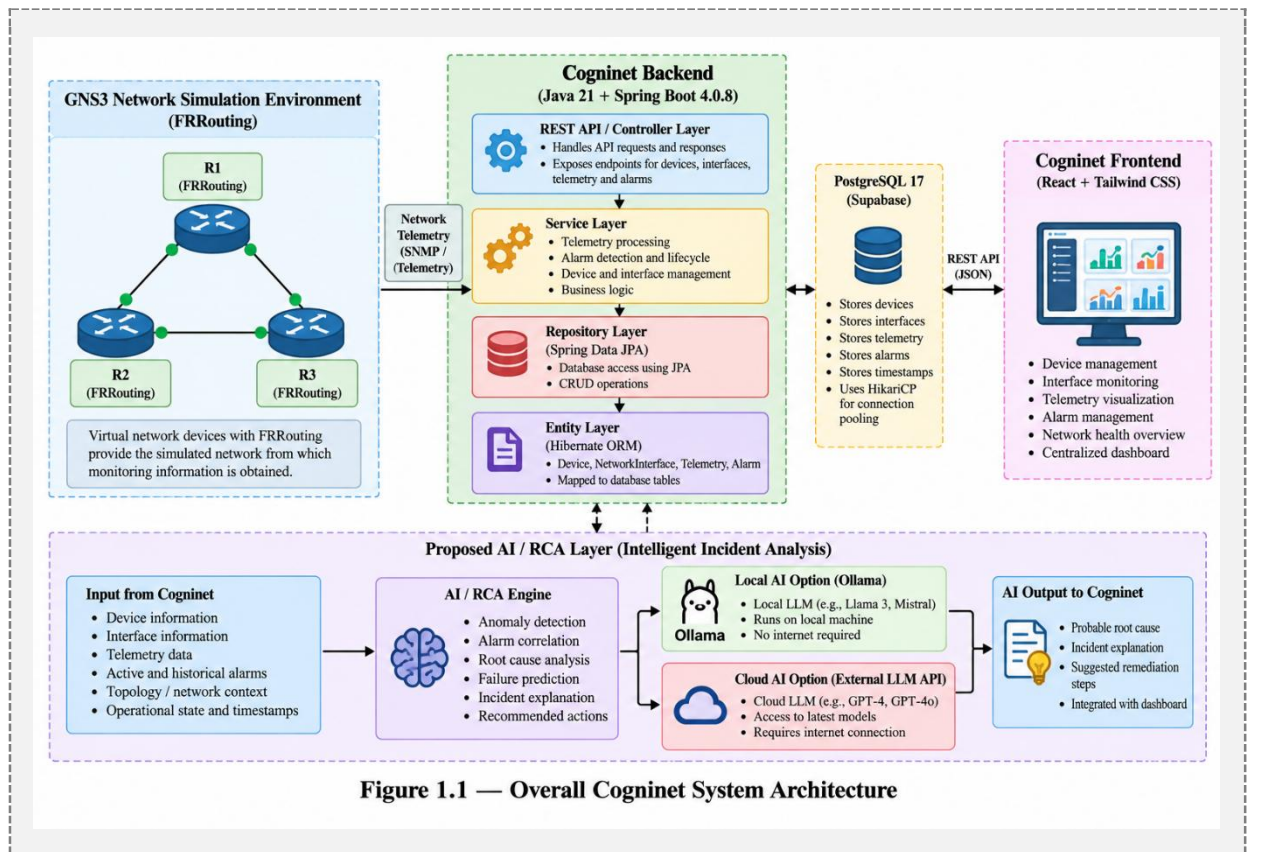


Figure 1.1 — Overall Cogninet System Architecture

Figure 2.1 — Overall Architecture of the Cogninet System Architecture

2.2 Network Simulation and Device Environment

The managed network is created in GNS3, a network simulation platform that allows virtual network devices to be created, connected and operated in a controlled environment [3]. FRRouting (FRR) is an open-source IP routing protocol suite that provides routing functionality for the virtual devices [4]. Together they supply a repeatable network from which monitoring information can be obtained without the use of physical equipment. The simulated environment contains three virtual routers, R1, R2 and R3, which are the managed devices of the current stage. Because the network is simulated, interface and device conditions

can be changed deliberately, for example by taking an interface out of service, and the resulting behaviour can be observed through the NMS. This makes it possible to exercise the telemetry, fault-detection and alarm functions of Cogninet under controlled conditions. The exact topology, links, interface names, routing protocols and addressing of the simulated network are not enumerated in this report; they are to be presented in Figure 2.2 once the actual project diagram is inserted.

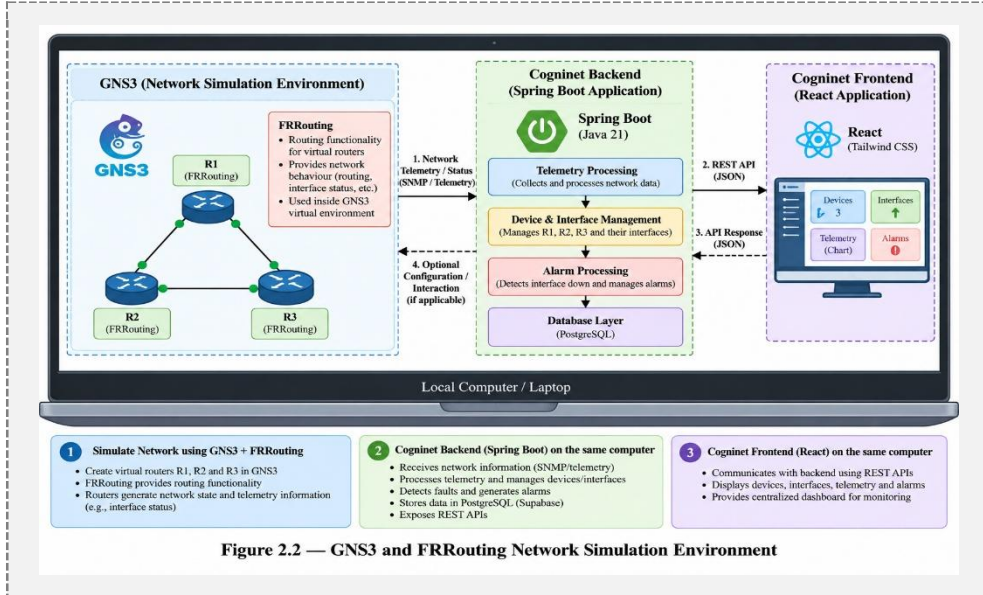


Figure 2.2 — GNS3 and FRRouting Network Simulation Environment

2.3 Network Device and Interface Management

Device and interface management are carried out through the layered backend that is described in Section 2.6 and summarised in Figure 2.3. The layers separate the receipt of requests, the application logic, the data access and the database, and they are applied uniformly to the Device and NetworkInterface entities as well as to telemetry and alarms.

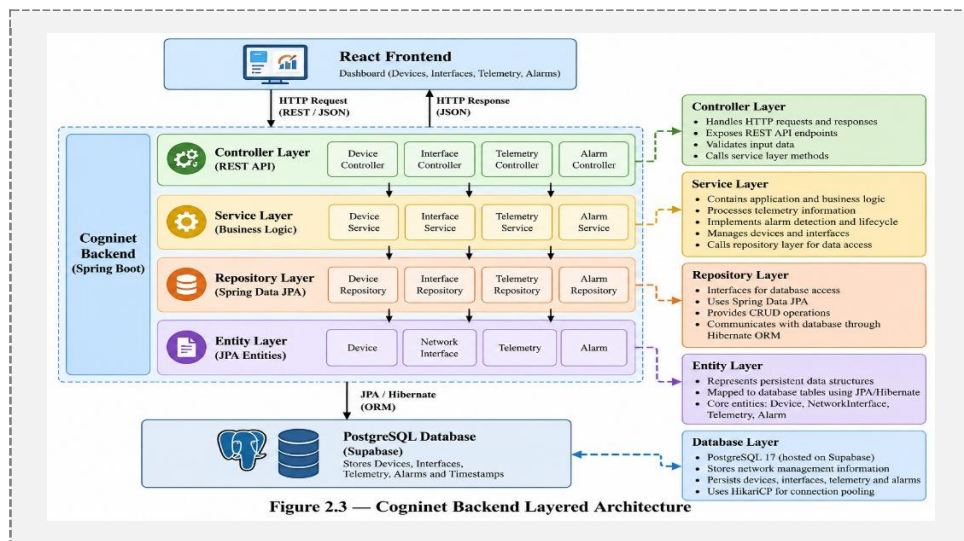


Figure 2.3 — Cogninet Backend Layered Architecture

The **Device** entity represents a managed network device such as R1, R2 or R3. At the conceptual level it captures the identity of the device, its hostname, its IP address and its

status, and it is associated with the interfaces that belong to it. The **NetworkInterface** entity represents an interface of a device. Conceptually it captures the identity of the interface, its name, its IP address where applicable, the device with which it is associated and its operational status. A device normally has several interfaces, whereas each interface belongs to exactly one device, so the relationship is one-to-many:

Device (1) — (many) NetworkInterface

This relationship is the basis for associating telemetry and alarms with the correct source. A measurement or fault is meaningful only when it can be attributed to a particular device and, where relevant, to a particular interface of that device. The core entities of the system and their roles are summarised in Table 2.2.

Table 2.2 — Core System Entities

Entity	Conceptual role	Conceptual relationship
Device	Represents a managed network device (for example R1, R2 or R3): identity, hostname, IP address and status.	Owens many NetworkInterface records.
NetworkInterface	Represents an interface of a device: identity, name, IP address where applicable and operational status.	Belongs to one Device.
Telemetry	Represents timestamped operational observations such as interface status, device status, CPU, memory and traffic/performance metrics.	Associated with a device and, where applicable, an interface.
Alarm	Represents a detected fault condition and its lifecycle, with severity, status, message, source and timestamps.	Associated with the device and interface in which the fault was detected.

2.4 Telemetry Collection and Processing

Telemetry provides the operational information on which every other NMS function depends. The information of interest includes interface operational status, device status, CPU utilisation, memory utilisation, interface traffic and performance metrics, and the time at which each observation was made. At the conceptual level the monitoring layer of Cogninet is referred to as SNMP / Telemetry, in recognition of the role of SNMP [1][6] as a standard means of obtaining management information from network devices. This report does not assert a specific collection mechanism beyond this conceptual description.

Telemetry is handled by the Cogninet backend in the following sequence:

1. **Receive.** The backend receives the monitoring information produced for a network device.
2. **Identify.** The device, and where applicable the interface, to which the observation belongs is identified.
3. **Validate and process.** The received information is checked and processed so that it can be used consistently.

4. **Store.** The processed telemetry is stored in the PostgreSQL database, preserving the association with the device and interface.
5. **Evaluate.** Fault conditions are evaluated from the processed telemetry and passed to the alarm logic.

The end-to-end flow is therefore Network Device → Monitoring / Telemetry → Cogninet Backend → Processing → Device / Interface Association → Database → Dashboard. Interface state is represented by the value of `interface_oper_status`, which is interpreted as follows and is applied consistently throughout the system:

- `interface_oper_status = 1` → the interface is **UP**.
- `interface_oper_status = 0` → the interface is **DOWN**.

The continuous observation made possible by telemetry supports fault detection, because a change in the operational status of an interface can be recognised as soon as it is reported rather than when an operator next inspects the device.

2.5 Alarm Management and Lifecycle

Alarm management converts detected faults into records that can be tracked until the fault is resolved. The lifecycle followed by an alarm is: Normal → Fault Detected → Alarm Created → Active / Updated → Fault Resolved → Alarm Cleared. The states are described in Table 2.3.

Table 2.3 — Alarm Lifecycle States

State	Description
Normal	No fault condition is detected.
Fault Detected	Telemetry indicates an abnormal condition, for example <code>interface_oper_status = 0</code> .
Alarm Created	A new alarm is created for the detected condition.
Active / Updated	Repeated observations update the existing alarm instead of creating duplicate alarms.
Cleared	The fault condition is resolved and the alarm is cleared.
Reopened	A previously cleared condition becomes active again; conceptually the alarm moves from Cleared back to Active.

The key interface-related fault is the interface-down condition, detected when `interface_oper_status = 0`. For this condition the alarm identifier is constructed deterministically as:

IF-DOWN-{deviceId}-{interfaceId}

Because the identifier is determined entirely by the device and the interface, every observation of the same fault on the same interface maps to the same alarm. This allows the system to recognise that an active alarm already exists and to update it, which prevents duplicate active alarms for the same device/interface fault condition. When the interface returns to the UP

state (`interface_oper_status = 1`), the alarm is cleared. If the same condition later becomes active again, the previously cleared condition is reopened, that is, it returns from the Cleared state to the Active state.

Each alarm carries a severity, a status, a message and a source. Its history is described by four timestamps: **firstSeenAt**, the time at which the condition was first observed; **lastSeenAt**, the time of the most recent observation; **clearedAt**, the time at which the condition was cleared; and **createdAt**, the time at which the alarm record was created. Together they allow the operator to determine how long a fault has existed, whether it is still being reported and when it was resolved.

Alarm Deduplication versus AI Alarm Correlation

Two concepts that are easily confused must be kept distinct. **Alarm deduplication** is a deterministic function of the core NMS. It prevents several active alarms from representing the same exact condition. If the same interface remains DOWN, the existing active alarm is updated on each new observation instead of a new alarm being created. **AI alarm correlation** is a proposed function of the AI/RCA extension. It attempts to identify relationships among multiple different alarms or events and to decide whether they belong to the same broader network incident, so that Multiple Related Alarms → Correlation → Common Incident → Probable Root Cause. Deduplication therefore operates on repeated reports of one condition, whereas correlation operates on distinct conditions that may share a cause. The two are not interchangeable, and the presence of deduplication does not imply that correlation is available.

2.6 Backend and REST API Architecture

The backend of Cogninet is implemented in Java 21 [7] using Spring Boot 4.0.8 [8]. Data access uses Spring Data JPA [9] with Hibernate ORM [10] as the object-relational mapping implementation. The project is built and managed with Maven, and the application runs on Apache Tomcat 11 [11]. Communication with the frontend uses REST/JSON APIs. The complete technology stack is given in Table 2.1.

The backend follows the layered architecture Controller → Service → Repository → Database. The **Controller** layer is the REST/HTTP interface of the system: it receives requests and returns JSON responses. The **Service** layer contains the business logic, including device operations, interface operations, telemetry processing and alarm logic. The **Repository** layer performs the persistence and data-access operations. The **Entity** classes form the domain model that is mapped to the database, and the **Database** layer is PostgreSQL 17. This separation keeps request handling, business rules and data access independent of one another, so that each can be modified or tested without disturbing the others. Specific endpoint paths, controller names and method names are not reproduced in this report.

2.7 Database Design

Cogninet uses PostgreSQL 17 [12] as its relational database, hosted on Supabase [13]. Database connections are managed by the HikariCP connection pool [14], and the application accesses the data through Spring Data JPA and Hibernate ORM. The persisted entities are Device, NetworkInterface, Telemetry and Alarm.

A relational database is appropriate for this data for several reasons. Device records and interface records are related through a well-defined one-to-many relationship that a relational model represents naturally. Telemetry records and alarm records refer to devices and interfaces and must remain consistent with them. Alarm and telemetry history is needed to understand how a condition developed, which requires durable, queryable storage of timestamped records. Finally, the transactional guarantees of a relational database help to keep the alarm state consistent when the same condition is reported repeatedly. The database schema columns are not reproduced in this report; the conceptual data model is given in Appendix C.

2.8 Frontend and Dashboard

The frontend is a React [15] application styled with Tailwind CSS [16]. It communicates with the backend through REST/JSON APIs and is intended to provide centralised views of the network. The views are the dashboard, which summarises network status; the devices view; the interfaces view; the telemetry view; and the alarms view. Each view presents information that the backend has already processed and stored, so the frontend does not itself evaluate faults.

A **Proposed AI Incident Analysis View** is defined as a future addition to the dashboard. It is intended to present, for an incident, a summary, the affected device, the affected interface, the probable root cause, the supporting evidence, the correlated alarms, the impact and the recommended action. This view is proposed and is shown as a placeholder in Figure 3.5.

2.9 AI/RCA Architecture

The AI/RCA layer is an extension of the Cogninet NMS and is described in this section as a proposed architecture. It is organised around five target capabilities, which are summarised in Table 2.4. All five are proposed capabilities, and none is claimed to have been experimentally validated.

Table 2.4 — AI Capabilities and Purpose

AI capability	Input	Purpose	Status
Anomaly Detection	Telemetry	Identify unusual behaviour	Proposed / Extension
Root Cause Analysis	Telemetry + alarms + topology	Identify the probable cause	Proposed / Extension
Alarm Correlation	Multiple related events	Reduce incident noise	Proposed / Extension
Failure Prediction	Historical telemetry	Identify potential future failure	Proposed / Extension
Incident Explanation	Structured incident context	Explain the incident to the operator	Proposed / Extension

Design Principle

The LLM is not presented as the fault detector. The proposed processing sequence is: Telemetry / Alarms → Preprocessing / Validation → Deterministic NMS Checks → Anomaly Detection / Event Correlation → Topology-Aware RCA → Structured Incident Context → LLM Explanation Layer → Operator Recommendations. Deterministic NMS logic handles reliable operational alarm detection; AI and machine-learning techniques identify patterns and anomalies; topology-aware analysis supports root cause reasoning; the LLM explains structured technical evidence; and the operator validates the resulting recommendations.

A. Anomaly Detection

The AI layer is proposed to analyse telemetry in order to identify unusual network behaviour. Potential signals include unusual traffic patterns, unexpected interface changes, abnormal CPU utilisation, abnormal memory utilisation and unusual telemetry trends. Anomaly detection techniques have been surveyed extensively [17], and classical machine-learning libraries such as Scikit-learn [18] are candidates for implementing them. The suitability of any technique for Cogninet telemetry has yet to be evaluated.

B. Root Cause Analysis

The AI/RCA engine is proposed to use structured network context to identify the probable root cause of an incident. The potential context comprises the device, the interface, telemetry, active alarms, alarm history, topology, routing or network state where available, and historical observations. Root cause analysis attempts to distinguish the **root cause** from the **downstream symptoms** that it produces. The analysis is intended to indicate a probable cause with supporting evidence; it is not claimed to be perfect or fully automated.

C. Alarm Correlation and Noise Reduction

A large network incident can produce multiple related events. The AI/RCA layer is proposed to correlate related events and to group them into a common incident, following the flow Multiple Related Alarms → Event Correlation → Incident Group → Probable Root Cause → Operator Explanation. As described in Section 2.5, this is distinct from the deterministic alarm deduplication performed by the core NMS.

D. Failure Prediction

Failure prediction is an advanced future capability. It may use historical telemetry, trends, degradation patterns, repeated faults and previous incidents to identify potential failure conditions before complete failure occurs. It has not been experimentally validated. Its realisation requires sufficient historical data, feature engineering, model development, training, validation and controlled experiments.

E. Incident Explanation

A Large Language Model [19][20] can transform structured incident information into an explanation that the network operator can read. The explanation is intended to answer what happened, which device and interface are affected, what evidence supports the incident, what the probable root cause is, what the potential impact is and what the operator should investigate next. Because the usefulness of an automated decision depends on the operator being able to understand it [21], the LLM is presented as an explanation and decision-support

layer. It is not the sole source of truth for network state, and the explanation must be grounded in the telemetry, alarms and structured network information supplied to it.

Provider-Independent Architecture

The AI/RCA layer is designed to be independent of any one AI provider. The principle is stated as follows.

“The Cogninet system uses a provider-independent AI/RCA layer so that incident analysis can be performed using either a locally hosted LLM through Ollama or an external LLM service through an authenticated API. The same structured incident context can be supplied to the selected provider without redesigning the core NMS.”

Two provider options are proposed. In **Option 1 – Local AI**, the flow is Cogninet NMS → AI/RCA Engine → Ollama [22] → Local LLM. This option executes locally, requires no external API key, is suitable for privacy-sensitive network information, can run within a local Ubuntu environment and allows the model to be selected locally. In **Option 2 – Cloud AI**, the flow is Cogninet NMS → AI/RCA Engine → External LLM API → API Key Authentication → Cloud LLM. Services such as OpenAI, Google Gemini or Anthropic Claude are mentioned only as examples of the kind of external provider that could be used; this report does not state that any of them has been integrated.

The input supplied to the AI/RCA engine is a structured context consisting of device information, interface information, telemetry, active alarms, alarm history, network topology, routing or network state where available, and historical observations. The proposed technology stack for this layer comprises Python for AI/ML processing, Pandas and NumPy for data processing, Scikit-learn for classical machine learning, Ollama for the local LLM, an external LLM API for cloud AI, and REST for integration. This is a **Proposed AI Technology Stack**, and its components are not stated to be implemented. The resulting architecture is shown in Figure 2.4

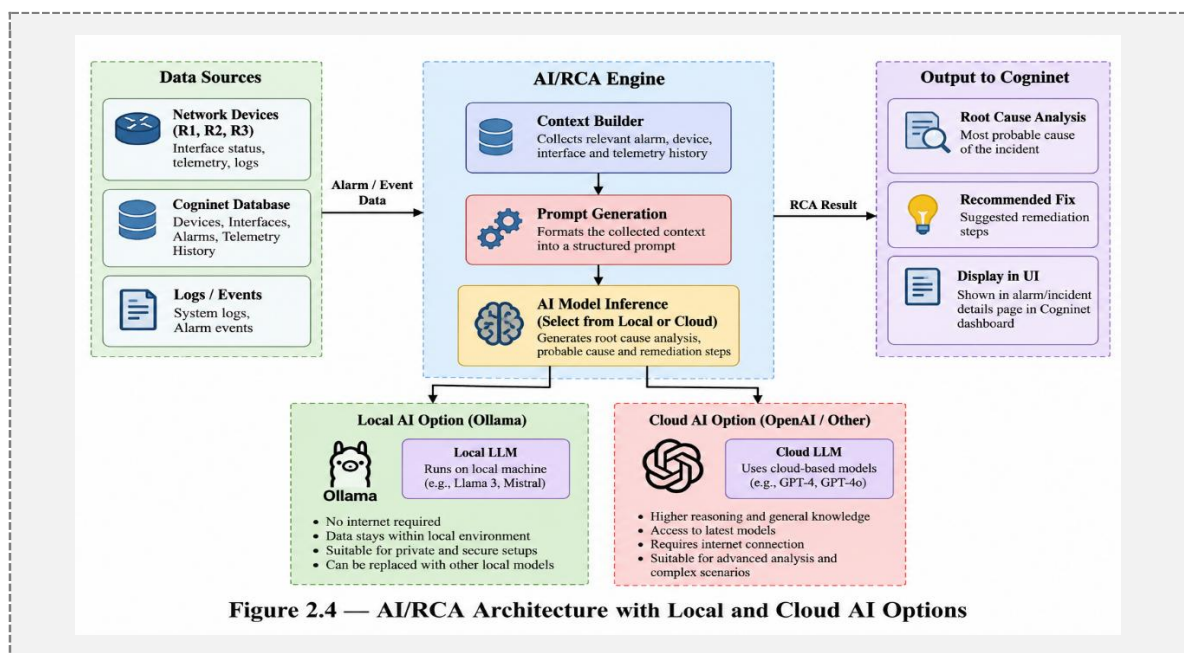


Figure 2.4— AI/RCA Architecture with Local and Cloud AI Options

2.10 Complete System Architecture

The complete architecture of Cogninet, shown in Figure 2.1, consists of eight layers.

Layer 1 – Network Environment. GNS3 and FRRouting provide the virtual network devices R1, R2 and R3.

Layer 2 – Monitoring / Telemetry. SNMP / Telemetry supplies device status, interface status, CPU, memory and traffic/performance information.

Layer 3 – Cogninet Backend. The Java 21 and Spring Boot 4.0.8 backend contains the REST APIs, controllers, services, repositories, telemetry processing and the alarm engine.

Layer 4 – Database. PostgreSQL 17 on Supabase, with HikariCP, stores devices, network interfaces, telemetry, alarms and historical information.

Layer 5 – AI / RCA Extension (proposed). Anomaly detection, root cause analysis, alarm correlation, failure prediction and incident explanation.

Layer 6 – AI Provider (proposed). Either Ollama with a local LLM, or an external LLM API with an API key.

Layer 7 – Frontend. React and Tailwind CSS provide the dashboard, devices, interfaces, telemetry and alarms views, together with the proposed AI incidents view.

Layer 8 – Network Operator. The operator receives network state and, through the proposed extension, the incident explanation, probable root cause, supporting evidence, impact information and recommended action.

Layers 1 to 4 and 7 constitute the core NMS. Layers 5 and 6 constitute the proposed AI extension, and the operator in Layer 8 remains responsible for validating every recommendation.

2.11 Implementation Flow

The implementation follows the steps below. Steps 12 to 15 belong to the proposed AI extension.

1. Configure the GNS3 network.
2. Configure the FRRouting devices.
3. Identify devices and interfaces.
4. Collect monitoring and telemetry information.
5. Process telemetry through the Cogninet backend.
6. Associate telemetry with devices and interfaces.
7. Store information in PostgreSQL.
8. Evaluate fault conditions.
9. Create, update and clear alarms.
10. Expose information through REST APIs.
11. **(Proposed)** Prepare structured incident context.

12. **(Proposed)** Pass the context to the AI/RCA engine.
13. **(Proposed)** Select the local Ollama provider or a cloud provider.
14. **(Proposed)** Return analysis and recommendations to the operator.

2.12 Methodology Summary

The methodology is summarised by layer in Table 2.5.

Table 2.5 — Methodology Summary by Layer

Layer	Components	Responsibility
Network Layer	GNS3, FRRouting, R1, R2, R3	Provide a controlled virtual network of managed devices.
Monitoring Layer	SNMP / Telemetry (conceptual)	Supply device status, interface status, CPU, memory and traffic/performance information.
Backend Layer	Java 21, Spring Boot 4.0.8, Spring Data JPA, Hibernate ORM, Maven, Tomcat 11	Process telemetry, apply business logic and expose REST/JSON APIs.
Database Layer	PostgreSQL 17, Supabase, HikariCP	Persist devices, interfaces, telemetry, alarms and history.
Alarm Layer	Alarm engine, IF-DOWN- <code>{deviceId}</code> - <code>{interfaceId}</code>	Detect interface-down faults and manage the alarm lifecycle without duplicates.
Frontend Layer	React, Tailwind CSS	Present devices, interfaces, telemetry and alarms to the operator.
AI Layer (proposed)	AI/RCA engine; Ollama or external LLM API	Provide anomaly detection, RCA, correlation, prediction and incident explanation.

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Current Implementation Status

Table 3.1 records the status of each component, using only the status that is supported by the project information.

Table 3.1 — Current Implementation Status

Component	Status	Description
GNS3 / FRRouting	Core project component	Controlled virtual network containing the routers R1, R2 and R3.
Device Management	Core project component	Management of devices through the Device entity.
Interface Management	Core project component	Management of interfaces through the NetworkInterface entity, associated with devices.
Telemetry	Core project component	Processing of operational observations and their association with devices and interfaces.
PostgreSQL	Core project component	PostgreSQL 17 on Supabase with HikariCP, accessed through JPA and Hibernate.
REST API	Core project component	REST/JSON interface of the Spring Boot backend.
Alarm Engine	Core project component	Interface-down detection and alarm lifecycle management.
React Dashboard	Core project component	React and Tailwind CSS views of devices, interfaces, telemetry and alarms.
AI/RCA	Proposed extension	Anomaly detection, RCA, alarm correlation, failure prediction and incident explanation, using a local or cloud LLM.

3.2 Network Device Management

Centralised device management gives the operator a single place in which the managed devices are recorded. Each device is held as a Device entity that carries its identity, hostname, IP address and status, so that R1, R2 and R3 can be viewed together instead of being inspected through separate consoles. Because every interface, telemetry record and alarm is associated with a device, the device record also acts as the anchor for the rest of the information held about the network. The device view of the dashboard is intended to present this information and is represented in Figure 3.1.

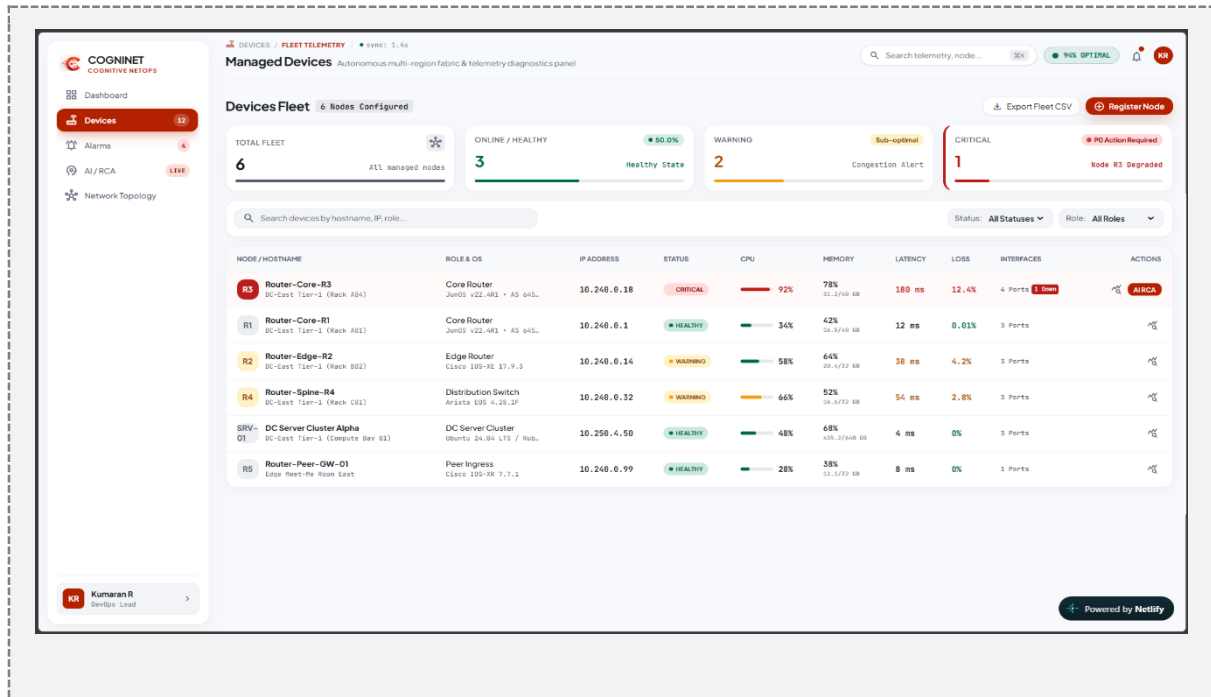


Figure 3.1 — Cogninet Device Management Dashboard

3.3 Interface Monitoring

Interface monitoring extends visibility from the device to the individual interface. A NetworkInterface record holds the interface identity, name, IP address where applicable, associated device and operational status. The operational status is derived from interface_oper_status, where 1 means UP and 0 means DOWN. Since interface state is the condition on which the interface-down alarm depends, accurate and consistent interpretation of this value is essential, and the same mapping is used by the telemetry processing, the alarm logic and the dashboard. The interface monitoring view is represented in Figure 3.2.

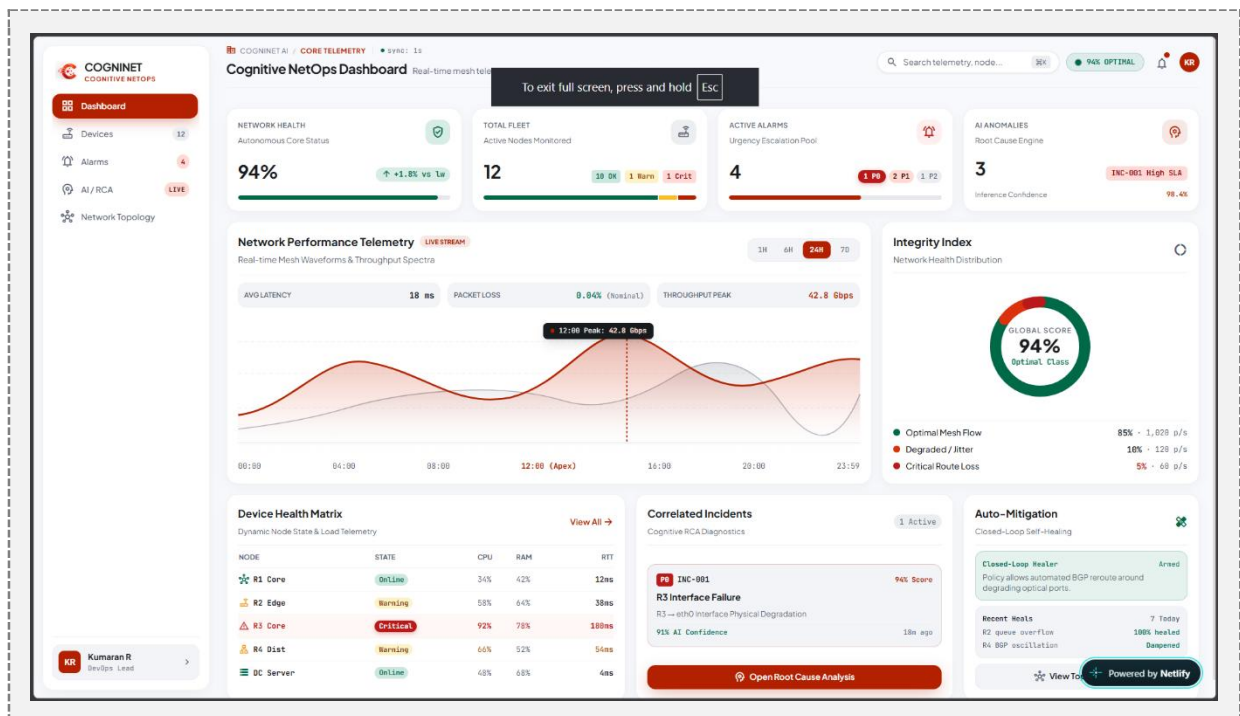


Figure 3.2 — Cogninet Interface Monitoring Dashboard

3.4 Telemetry Processing

Telemetry processing connects the simulated network to the rest of the NMS. Each observation is received, attributed to a device and interface, validated and processed, stored in PostgreSQL and then evaluated for fault conditions. Storing the observations with their timestamps preserves a history of the network, which is useful for the operator and is also the data on which any later anomaly detection or failure prediction would depend. The quality of this information determines the quality of every function built on it, which is why the association between telemetry and the correct device and interface is treated as a central step. The telemetry view is represented in Figure 3.3.

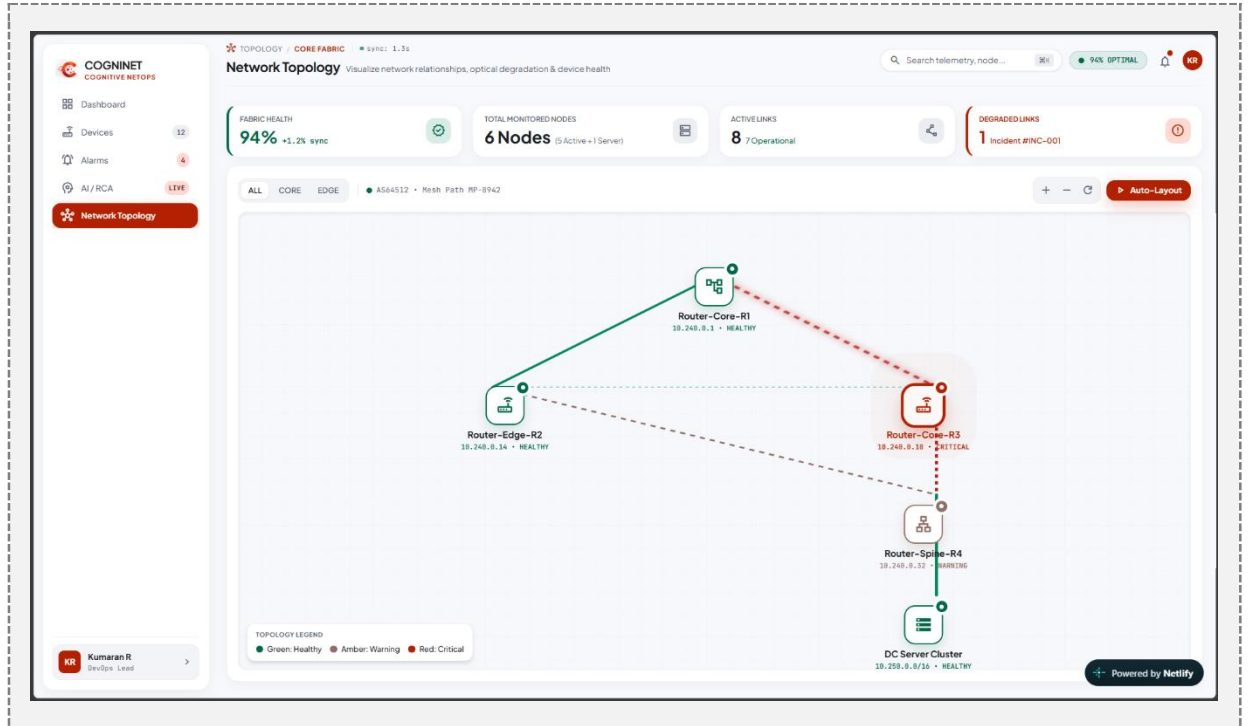


Figure 3.3 — Cogninet Telemetry Dashboard

3.5 Alarm Detection and Lifecycle

The alarm engine applies deterministic rules to processed telemetry. When `interface_oper_status = 0` for an interface, an interface-down fault is detected and an alarm is created with the identifier `IF-DOWN-{deviceId}-{interfaceId}`. While the interface remains DOWN, each further observation updates the existing active alarm, so that only one active alarm exists for the condition and `lastSeenAt` reflects the latest observation. When `interface_oper_status` returns to 1, the alarm is cleared and `clearedAt` records the time of clearing. If the same condition occurs again, the previously cleared condition is reopened and returns from the Cleared state to the Active state. The timestamps `firstSeenAt`, `lastSeenAt`, `clearedAt` and `createdAt` together describe the history of the condition.

This behaviour illustrates the practical difference between the deterministic and the proposed capabilities of the system. The alarm engine can state reliably that a particular interface is DOWN, that an alarm exists for it and whether the alarm is active or cleared. It cannot, by itself, state that the alarms on several interfaces share a single cause; that is the role of the proposed correlation and RCA functions. The alarm view is represented in Figure 3.4.

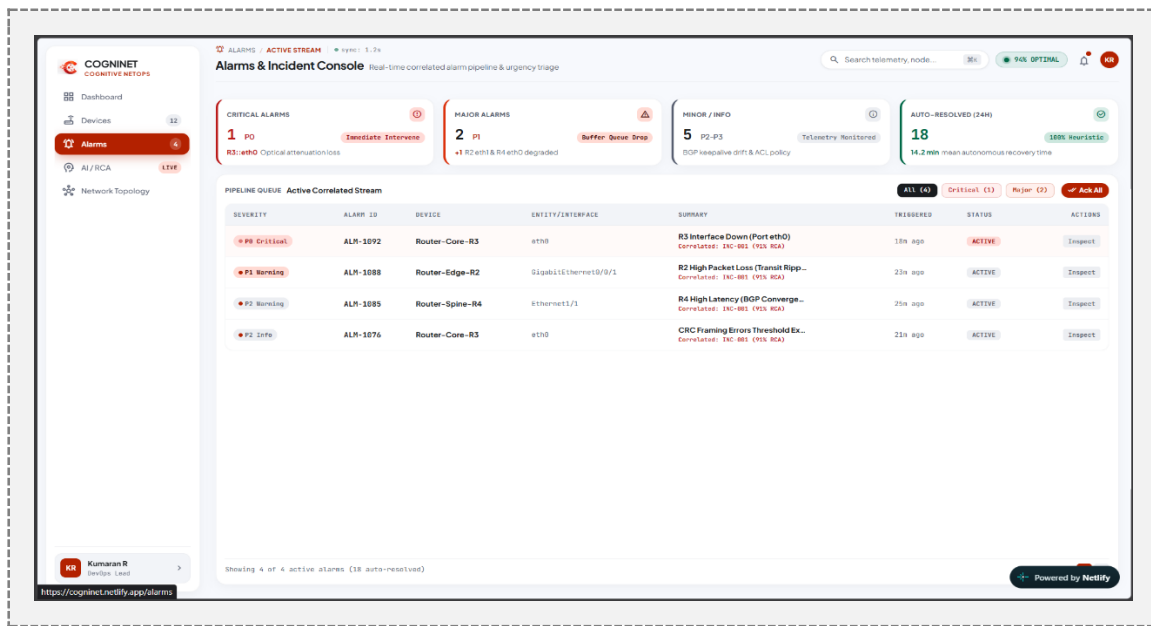


Figure 3.4 — Cogninet Alarm Dashboard

3.6 Backend API and Database Integration

The backend and the database are integrated through the layered structure described in Chapter 2. Controllers receive requests and return JSON, services apply the business logic, and repositories access PostgreSQL through Spring Data JPA and Hibernate ORM. HikariCP manages the pool of database connections, so that the backend does not need to establish a new connection for every operation. This arrangement gives the frontend a single, uniform route to the stored information and keeps the persistence concerns separate from the application logic. The responsibilities of the principal components are summarised in Table 3.2.

Table 3.2 — System Components and Responsibilities

Component	Responsibility
GNS3 and FRRouting	Supply the virtual network devices that are managed and monitored.
Controller layer	Receive requests and return JSON responses.
Service layer	Device and interface operations, telemetry processing and alarm logic.
Repository layer	Persistence and data-access operations.
PostgreSQL 17 on Supabase	Persistent storage of devices, interfaces, telemetry and alarms.
HikariCP	Management of database connections.
React and Tailwind CSS	Presentation of network state to the operator.
AI/RCA engine (proposed)	Analysis of structured incident context and preparation of explanations.
AI provider (proposed)	Local LLM through Ollama, or external LLM through an authenticated API.

3.7 Frontend Dashboard

The React and Tailwind CSS dashboard brings the information described above into one interface. It lets the operator move between the dashboard overview, devices, interfaces, telemetry and alarms without leaving the application, and it obtains all of its data from the backend through REST/JSON. Actual screenshots of the dashboard are shown in Figures 3.1 to 3.4 once they are supplied; no screenshots are reproduced in this draft, and none are simulated.

3.8 AI/RCA Extension

The proposed AI/RCA extension would use the data that the NMS already holds. Telemetry, alarms, topology and historical context are combined into a structured incident context and passed to the AI/RCA engine; the engine performs analysis, and the selected provider, either a local Ollama LLM or a cloud LLM API, produces an explanation, a probable root cause and recommendations for the network operator. The current NMS foundation provides the required data for this extension, but the extension itself requires further implementation and validation.

3.9 System-Level Discussion

The value of Cogninet lies in the combination of its parts. Monitoring and telemetry supply the observations, the alarm lifecycle turns observations into trackable fault records, centralised visualisation makes the state of the network visible in one place, and the proposed AI-assisted analysis would help the operator to interpret groups of related events

The distinction between deterministic NMS logic and AI/RCA logic, introduced in Chapter 2, is summarised in Table 3.3. The deterministic logic provides dependable detection and record-keeping. The AI/RCA logic provides interpretation and explanation on top of that record, and it is subordinate to it: the network state is always grounded in actual telemetry, alarms and structured network information, and the operator validates any recommendation before acting.

Table 3.3 — Deterministic NMS Logic versus AI/RCA Logic

Aspect	Deterministic NMS logic (core)	AI/RCA logic (proposed)
Functions	Telemetry evaluation, interface state, fault detection, alarm generation, alarm lifecycle, duplicate prevention.	Anomaly detection, event correlation, root-cause reasoning, prediction, explanation, recommendations.
Basis	Explicit rules, for example <code>interface_oper_status = 0</code> means DOWN.	Patterns, topology-aware reasoning and language-model explanation over structured context.
Role	Authoritative detection of operational faults.	Decision support for the operator; not an authority over the network.
Status	Core project component.	Proposed extension.

CHAPTER 4

CONCLUSION AND FUTURE WORK

4.1 Conclusion

This report has presented Cogninet, an intelligent Network Management System developed for the WIPRO NMS Capstone Project. Cogninet provides a centralised view of network devices, interfaces, telemetry and alarms. Its network environment is created with GNS3 and FRRouting, and its backend is a layered Spring Boot application that processes telemetry, applies deterministic alarm logic and exposes REST/JSON APIs. PostgreSQL 17 on Supabase, accessed through Spring Data JPA, Hibernate ORM and HikariCP, stores the information, and a React and Tailwind CSS dashboard presents it to the operator. The alarm management function turns interface state into manageable alarms. With `interface_oper_status = 1` meaning UP and 0 meaning DOWN, an interface-down fault produces an alarm identified by `IF-DOWN-{{deviceId}}-{{interfaceId}}`. Repeated reports of the same fault update that alarm rather than duplicating it, the alarm is cleared when the fault is resolved, and the same condition can be reopened if it recurs. The report also defined a proposed, provider-independent AI/RCA extension that adds anomaly detection, root cause analysis, alarm correlation, failure prediction and incident explanation, using either a local Ollama LLM or a cloud LLM API. By keeping detection deterministic and using the AI layer for analysis and explanation, the architecture allows Cogninet to evolve toward intelligent network management without compromising the reliability of the core system.

4.2 Future Work

Network management foundation. Additional telemetry types and additional alarm types can be added to extend fault coverage beyond the interface-down condition, and topology discovery can supply the network structure that RCA requires. Authentication, authorisation and logging should be added, and scalability and production testing should be carried out.

Data and models. Historical datasets should be collected from the NMS. They would support machine-learning anomaly detection and failure prediction, which require feature engineering, training, validation and controlled experiments. Model evaluation must be carried out and reported with measured results before any claim of accuracy is made.

AI integration. Advanced alarm correlation should be implemented and Ollama and cloud LLM integration should be developed behind the provider-independent layer. An AI incident dashboard should be built that displays the evidence and the confidence of each analysis.

Operator feedback. AI feedback mechanisms should allow operators to confirm or correct the analysis, so that the quality of the AI layer can be assessed and improved over time.

4.3 Final Perspective

Conventional network management establishes what is happening in a network; intelligent network management also helps to explain why. The Cogninet architecture provides a foundation for this progression by combining conventional network monitoring and fault management with a provider-independent AI/RCA layer. Its deterministic core supplies

REFERENCES

- [1] J. Case, M. Fedor, M. Schoffstall, and J. Davin, “A Simple Network Management Protocol (SNMP),” IETF, RFC 1157, May 1990.
- [2] K. McCloghrie and M. Rose, “Management Information Base for Network Management of TCP/IP-based internets: MIB-II,” IETF, RFC 1213, Mar. 1991.
- [3] GNS3 Technologies Inc., “GNS3 Documentation,” official project documentation (URL and access date to be added by the authors).
- [4] FRRouting Project, “FRRouting User Guide,” official project documentation (URL and access date to be added by the authors).
- [5] M. Steinder and A. S. Sethi, “A survey of fault localization techniques in computer networks,” *Science of Computer Programming*, vol. 53, no. 2, pp. 165–194, 2004.
- [6] D. Harrington, R. Presuhn, and B. Wijnen, “An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks,” IETF, RFC 3411, Dec. 2002.
- [7] Oracle Corporation, “Java Platform, Standard Edition 21 Documentation,” official documentation (URL and access date to be added by the authors).
- [8] Spring Project, “Spring Boot Reference Documentation,” official project documentation (URL and access date to be added by the authors).
- [9] Spring Project, “Spring Data JPA Reference Documentation,” official project documentation (URL and access date to be added by the authors).
- [10] Hibernate Project, “Hibernate ORM Documentation,” official project documentation (URL and access date to be added by the authors).
- [11] Apache Software Foundation, “Apache Tomcat 11 Documentation,” official project documentation (URL and access date to be added by the authors).
- [12] PostgreSQL Global Development Group, “PostgreSQL 17 Documentation,” official project documentation (URL and access date to be added by the authors).
- [13] Supabase, “Supabase Documentation,” official project documentation (URL and access date to be added by the authors).
- [14] HikariCP Project, “HikariCP: JDBC connection pool,” official project repository and documentation (URL and access date to be added by the authors).
- [15] Meta Platforms, Inc., “React Documentation,” official project documentation (URL and access date to be added by the authors).
- [16] Tailwind Labs, “Tailwind CSS Documentation,” official project documentation (URL and access date to be added by the authors).
- [17] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, Art. no. 15, 2009.
- [18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

APPENDICES

APPENDIX A — End-to-End Processing Flow

The end-to-end processing flow of Cogninet is as follows. Steps 11 to 13 belong to the proposed AI extension.

1. Network devices operate in GNS3 / FRRouting.
2. Monitoring information is collected.
3. Device and interface context is identified.
4. Telemetry is processed.
5. Telemetry is stored in PostgreSQL.
6. Fault conditions are evaluated.
7. An alarm is created or updated.
8. The alarm is cleared when the fault resolves.
9. REST APIs expose the information.
10. The React dashboard presents the network state.
11. **(Proposed)** AI/RCA context is prepared.
12. **(Proposed)** The AI provider, local or cloud, analyses the incident.
13. **(Proposed)** Explanation and recommendations are returned to the operator.

APPENDIX B — Key Terms

Table B.1 — Key Terms

Term	Meaning
NMS	Network Management System: a system for monitoring and managing network devices, faults and performance.
GNS3	A network simulation platform used to create and operate virtual network environments.
FRRouting / FRR	An open-source IP routing protocol suite providing routing functionality.
SNMP	Simple Network Management Protocol: a standard protocol for obtaining management information from network devices.
Telemetry	Timestamped operational measurements and status information obtained from the network.
REST	Representational State Transfer: an architectural style for web APIs.
API	Application Programming Interface: a defined interface through which software components communicate.
Spring Boot	A Java framework for building stand-alone application and REST services.
JPA	Java Persistence API: a specification for object-relational persistence in Java.
Hibernate	An object-relational mapping framework that implements JPA.
PostgreSQL	An open-source relational database system.

Term	Meaning
Supabase	A platform that provides hosted PostgreSQL databases.
React	A JavaScript library for building user interfaces.
Tailwind CSS	A utility-based CSS framework for styling user interfaces.
AI	Artificial Intelligence.
ML	Machine Learning: methods that learn patterns from data.
LLM	Large Language Model: a language model trained on large text collections.
Ollama	A tool for running large language models locally.
Anomaly Detection	Identification of observations that deviate from normal behaviour.
Alarm Correlation	Grouping of related alarms or events into a common incident.
RCA / Root Cause Analysis	Determination of the underlying cause of an incident as distinct from its downstream symptoms.
Network Topology	The arrangement of devices and links in a network.

APPENDIX C — Core Data Model

The core data model consists of four entities. It is described conceptually; exact database columns are not reproduced.

Table C.1 — Core Data Model

Entity	Conceptual content	Conceptual relationships
Device	Device identity, hostname, IP address and status.	One Device has many NetworkInterface records.
NetworkInterface	Interface identity, interface name, IP address where applicable, associated device and operational status.	Many NetworkInterface records belong to one Device.
Telemetry	Timestamped observations such as interface operational status (interface_oper_status), device status, CPU, memory and traffic/performance metrics.	Each observation relates to a device and, where applicable, an interface.
Alarm	Alarm identifier (for example IF-DOWN-{deviceId}-{interfaceId}), severity, status, message, source and the timestamps firstSeenAt, lastSeenAt, clearedAt and createdAt.	Relates to the device and interface in which the fault was detected.

Conceptually, a Device owns many NetworkInterface records; Telemetry is received for devices and interfaces and is evaluated by the alarm logic; and an Alarm records the resulting fault condition for the relevant device and interface.